PORTFOLIO · 一个人 · 2026

把事做成，
并让它可被验证。

我不是工程师。我是驱动 AI 干活的人。

四年工业岗位之后离职，现在一个人做。我不亲手画版图，也不自己跑每一条命令——那些反复重复的操作劳动全部交给 AI 和脚本。我负责三件交不出去的事：定目标与边界、判断交出来的东西是真是假、判断什么时候停。

下面这一件事，是我用来证明这三件事的。

1	7 代	6 道	1 条
完整闭环	迭代	质量门	命令
RTL → GDS	每代解决一个问题	每次提交重跑	外行也能证伪

1ouxilisi

github.com/1ouxilisi/sha256-openlane-tapeout

案例

一颗真的做出来了的 SHA-256 芯片

没有商业 EDA 授权，没有流片预算，没有制造排期。OpenLane 1.1.1 + SkyWater sky130A，全开源工具链，从一个 Verilog 文件做到能打开的 GDSII。

项目	结果
最终工作点	55.56 MHz（18 ns），三 PVT corner setup / hold 全正
面积 / 单元	0.4096 mm²（640 × 640 µm die）· 11,475 cells · 利用率 45.77%
物理验证	Magic 与 KLayout 两套 signoff DRC 均零违例（各 0 条；KLayout 218 个规则类，均不含天线 / 密度）；密度 deck 六层空孔率全部在阈值之上、0 条违例；两份 GDS 逐层 XOR，21 个制造层全一致；LVS 展平后 166,353 器件 / 84,178 网两侧相等，Circuits match uniquely（带口径）；天线这一项已修好（布线级）——原报 67 个 net / 87 个 pin 越限；补上我漏传的 -allow_congestion 后三轮插 430 个二极管、复测 0/0，新 GDS 18,618,660 字节，其上两套 DRC 与密度均 0 条；未在新版重跑功耗 / XOR；时序已补到 signoff 口径——OpenROAD rcx 从新版布线 DEF 直接提寄生（-lef_res，三份 RCX 规则各一份 SPEF），三个工艺角各算一次，最差 setup +0.4058 ns（Slowest）、hold +0.4096 ns（Fastest）、TNS 0，对比未修版 +0.5619 / +0.4021 只少 0.156 ns（同脚本在旧版复现 +0.5619，口径对齐；早前估算口径 +9.8329 → +9.9243 已作废）；LVS 已在新版跑通，两侧 15,588 器件 / 15,645 网相等
第 7 代做了什么	换掉消息调度结构（16 深循环窗口 → 移位寄存器），同 die 从 41.67 MHz 提到 55.56 MHz（+33%），单元数反而从 12,288 降到 11,475
被它推翻的话	项目早期文档写着「43.9 MHz 是这条 RTL 的上限」——那是旧架构的上限，不是工艺的上限

没做成的部分也在这页上。

v1–v4 时序没收敛（v4 nom 角 setup 余量 -7.63 ns，按签核标准不合格）；裸综合对照与 OpenLane 结论符号相反；功耗角名一度标错（6.84 mW 是 Typical 角，不是最保守的 Fastest）。这些不是丢人的地方，是判断力的来源。

凭什么可信

不是「我写它通过它就通过」

三条互不依赖的证据链。任一条断了，结论就站不住。

① 结构可信

仓库每次 push，在干净的 Ubuntu runner 上重跑 6 道门：v6 / v7 功能回归、独立向量校验、两代静态 lint、驱动级三方对照。任何人都能看到跑没跑过、绿没绿。

② 数字可信

期望摘要由 Python hashlib 独立计算，不用我自己写的常量；两个版图工具出的 GDS 逐层 XOR，不一致直接报错。「自己验自己」被结构性地禁掉。

③ 外人可证伪

不需要懂芯片，也不需要白皮书。三条命令，跑不通就是假的——

```
bash scripts/verify.sh            # ■■■■
bash scripts/verify_driver.sh      # C ■■ -> ■ RTL -> hashlib
python3 scripts/check_vectors.py   # ■■■■■■■■
```

最后一层是把 C 主机驱动打进验证链：C 驱动打出寄存器写序列 → Verilog 主机模型原样发到真 RTL 的 AXI4-Lite 总线上 → hashlib 算期望值。三份实现互不依赖，四条消息全过（含 119 字节两块流式）。这一层同时回答了「别人能不能接手」。

不想开终端也行：打开作品集首页的「在线自证」。两条互不依赖的实现（浏览器原生 + 页面自写的 512-bit 分块状态机）分别复算同一组向量，证据文件现算指纹、可与仓库对照，另有 200 组随机压测和一个「把常量改错一位」的现场演示，5 秒，不用装任何东西。

另外这一页上的每个数字都能点开看原文：「数字溯源」把 55.56 MHz、11,475 cells、0.4096 mm²、9.41 mW 和 setup / hold 的每一格，逐条追到 OpenLane 本次 run 的原文行与行号。

本次 run 在第 37 步（GDS 逐层 XOR）因 KLayout 运行时故障中断，这一步事后用同一套 KLayout 库的 Python 接口补做：42 个层里 21 个制造层几何完全一致，唯一差异是一个不参与制造的 flow 标记层，流水线本身也把它排除；同一轮里 Magic signoff DRC 也补跑成功，零条违例（不含天线与密度检查）；KLayout 侧的 signoff DRC 同样补跑成功——218 个规则类、0 条违例，而且我先把之前四次「必崩」的错误归因纠正了：tlFileUtils.cc:979 是 KLayout 找不到 /proc/<pid>/exe 时的断言，chroot 里 /proc 是空目录，把那个路径造出来再用同一个 pid exec 就一次通过；0 条不是空跑，故意造一个 met1 间距 0.10 µm 的小版图，同一 deck 立刻报 1 条 m1.2。

密度那一份 deck 也跑完了：六层空孔率 li1 0.4988、m1 0.7280、m2 0.9061、m3 0.9293、m4 0.9470、m5 0.9732，全部在阈值之上、0 条违例，同样带正对照——把 met1 铺满整个 die，同一 deck 立刻报 1 条 m1.pd.1d。

同一套 DRC deck 与密度 deck 在另一版 GDS（40,396,222 字节，ctsfix3 那次流片的签核产出，sha256 0cb148d6 开头）上又各跑了一次：KLayout DRC 218 个规则类、0 条，93.690 秒，正对照 ctl.gds（两条 met1 间距 0.10 µm）报 1 条 m1.2；密度六层 0.4993 / 0.7275 / 0.9055 / 0.9264 / 0.9460 / 0.9763，全在阈值之上、0 条，5.780 秒。

这版 0 条报告与上一版逐字节相同（34,074 字节），因为空报告只含类别表、不含任何条目——两次 run 只能靠日志里的输入路径与耗时区分，这一点连同「上游传 topcell / thread、deck 读的是 $top_cell / $thr」都写进了页面与证据说明，不靠哈希冒充证据。天线这一项也修好了，但只到「布线级」：原始状态用 OpenROAD check_antennas 补跑，报 67 个 net、87 个 pin 越限，本 run 的 GRT_REPAIR_ANTENNAS = 0、DiodeCells = 0，一个二极管都没插。

前五次重跑都被 DRT-0218 / GRT-0226 挡住，当时判成「工具在这份设计上走不通」——这个结论撤回了：真因是我手写脚本漏传 global_route 的 -allow_congestion（OpenLane 只在 GRT_ALLOW_CONGESTION=1 时才传这个开关，本 run 配置正是 1）。补上后从 22-global.odb 出发，三轮 GRT + repair_antennas 共插 430 个二极管，详细布线后复测 0 / 0，每轮布线 DRC 报告都是 0 字节。

补 fill 后流出新 GDS：18,618,660 字节；在新 GDS 上重跑 KLayout DRC（218 类 0 条）、密度（六层 0 条）、Magic DRC（No errors found）。

整条 flow 重跑的 ctsfix3 那版也把天线补齐了：复检先报 39 net / 49 pin（不是 0），随后在同一份布线后 ODB 上按「修天线 → 详细布局 → 全局布线 → 详细布线 → 复检」跑三轮，32/38 → 2/2 → 0/0，另起进程独立复检仍 0/0；补 fill 后重出 GDS 40,666,588 字节，在这一份上四项同版复跑：KLayout DRC 218 类 0 条、密度六层 0 条、LVS Circuits match uniquely、GDS XOR TOTAL_DIFF=0。

该版没有重跑 STA 与 DRV，所以 ctsfix3 的 setup / hold 数字不能顺延。边界也写在这里：新版只到布线级修好，功耗、GDS XOR、ERC 与 iRdrop 还没重跑，本页的功耗数字仍属未修版；LVS 则连新版也跑通了——同一版 GDS 提出的网表对 OpenROAD 网表，两侧 15,588 个器件 / 15,645 条网相等，149 个叶子单元逐个 Netlists match uniquely。

时序这一项已经补到 signoff 口径：寄生由 OpenROAD rcx 从新版布线 DEF 直接提（-lef_res，max/nom/min 三份 RCX 规则各写一份 SPEF），三个工艺角各算一次，最差 setup +0.4058 ns（Slowest）、最差 hold +0.4096 ns（Fastest）、TNS 0；对比未修版 +0.5619 / +0.4021，setup 只少 0.156 ns，18 ns 周期仍然成立。

同一套脚本在旧版产物上能复现 +0.5619，说明口径没换；早前那版估算口径（tt 单角 + estimate_parasitics -global_routing，+9.8329 → +9.9243）过于乐观，已作废。

但同一版上补查设计规则（DRV）没过：SDC 第 212–213 行要求 max_transition 0.75、max_fanout 10，三 corner 逐 pin 查下来新版 Slowest 有 255 条 slew、219 条 fanout 违例（最差 slew 1.17 ns、最差扇出 27），旧版同脚本也早有 210 / 143 条、最差扇出 25——所以准确说法是「时序通过、DRV 未清」，不是「signoff 通过」；要清得回头改时钟树结构，不能在 STA 里改约束糊过去。

根因也追到行号：config 第 10 行时钟扇出目标设 16、第 29 行 sink 聚类设 25，而第 207 行生成的 SDC 卡 10——配置自己跟自己对不上，中间没哪一步把时钟树压回 10 以内；旧版共用同一份 config，所以同源。改这两个数（降到 ≤10）重跑 CTS 才有可能清，本轮只定位没重跑。

后来真的跑了：从 placement 重跑 CTS，聚类目标压到 8，扇出违例 143 → 9（原配置重跑量到的 143，与线上 signoff 的 143 逐条相同），代价面积 +0.35%；剩下 9 条当时被我归因到工具内部「每簇最多 15 个 sink」的停止判据上（15 > 10），判成参数调不到 0。

这条错了，我撤回：把聚类换成 size 9、直径放到 70–100，扇出违例就是 0（最大扇出正好等于限值 10），面积 142,876 u² 反比原配置小 76 u²；重跑全局与详细布线（按 config 第 143 行 GRT_ADJUSTMENT 0.3 设层调整），详细布线 0 违例、走线 584,628 µm、过孔 104,134，布线后扇出仍是 0。

再用 RCX 三 corner 真 STA 复核这一版：setup +4.0576 / +11.0275 / +12.2001 三格全 MET，hold +0.4374 / +0.0820 / −0.0391（Fastest 角一条违例），max fanout 与 max capacitance 三格全 MET，max slew 在 Slowest 还剩 90 条（最差 1.04，限 0.75）。这一版是从 placement DEF 派生的，没有天线二极管、没跑 LVS，数字不能与上文 fixed16 逐项对比，那条 Fastest 的 hold 违例也没修——修掉扇出不等于 DRV 全清。

旧版这一步的归零过程，值钱的不是结论，是三次公开撤回——上一版把原因归到版图网表里 1,795 行实例没写单元名，后来自己推翻了这个说法（那是标准 SPICE 续行，续行拼回去 33,984 行全有单元名）；接着又推翻了一次「两侧端口表整体错位一格」的读法——两侧端口表本来就是 98 / 98 且逐字相同，把两侧头部折成每行 8 个、两个方向各跑一次，结果一字未变，而那 47 个名字在报告里只打印一遍（只有端口真缺时 netgen 才打印两遍），说明端口两侧都在、失败在它们所连的网这一层。现在只剩一句有依据的话：47 个端口所连的网没配上，机制未定。电子稿上写清了，包括撤回记录。

接着又比了一层：不看端口名、看端口连着什么——98 个端口里 94 个连到哪个器件的哪个脚都逐字相同，剩下 4 个是转换器漏读了网表末尾 4 条 assign 别名（补上重跑仍是 47），于是「版图与网表的实际连接差异」这句也撤了；最后把两侧网名做差，真正多出的只有 VNB / VPB 两个网——版图里井与衬底由 5,496 个 tap 直接接电源，而网表里这些 tap 引脚是空的被整批丢掉，成了悬空网。把这两个网并入电源重跑，47 行归零：展平后 166,353 个器件、84,178 条网两侧完全相等，netgen 原文 Circuits match uniquely。

这个「过」带口径——井/衬底并入电源、叶子单元由 netgen 展平后再比，所以写的是「顶层晶体管级等价」；另一条路（把 5,496 个 tap 补回网表）实跑过了，做不成：PDK 里 tap 单元只有 VGND/VPWR 两个脚、body 为空，补回后仍是 47 行，强加 VNB/VPB 脚仍是 47 行——所以口径不是可选项，是这个单元定义逼出来的唯一写法。层号已对到 PDK 自己的图层名表。

03 / BOUNDARY

边界

我负责什么，不负责什么

边界写清楚，比能力写得漂亮更有用。

我负责	我不负责
定目标、划边界、切里程碑	具体版图细节、每一行 RTL
判断交付物真假	把自己变成工程师
把重复劳动交给机器	亲手跑每条命令、盯每个日志
判断什么时候停	无限逼近「更完美」

可迁移

能搬走的不是知识，是验收结构

领域	证据载体
芯片	GDS + 多角 STA 报告 + DRC + 双工具 XOR
机器人动作包	关节轨迹 + 正运动学复算 + 真机/仿真同帧对比
短剧 / 内容	成片 + 分镜与成片逐镜一致 + 素材授权链
汽修手册	车型 / 年款 / 扭矩值可溯源 + 实车一步一验
财务分析	每格数字回原始报表 + 口径写明 + 可复算

怎么干活

一个人的五步，重复劳动全部外包

既然身份是「驱动 AI 干活的人」，该展示的就不是我会多少知识，而是我每次怎么组织一次工作。

1 定目标

一句话说清要什么。「做出一个能打开的 GDS」，不是「研究一下」——目标得能被别人复述。

2 划边界

同时说清不做什么：不量产、不做 DFT 扫链、不追完美。边界不写，工期就没有终点。

3 全权交出去

跑流程、读日志、写脚本、渲染、排版——重复劳动全交给机器。人只盯两件事：还剩什么、结论对不对。

4 判真假

只认互不依赖的证据，不认「我跑通了」。三条：独立复算、双工具互验、外人能一条命令证伪。

5 决定停还是继续

每代只改一个变量，改完先判断收益，再决定要不要下一代。不靠惯性往前推。

三个真实事例，都在同一件事里

当时以为	实际是
功耗 6.84 mW 取的是最保守的 Fastest 角	其实是 Typical 角。数字没错，标签错了——比最坏情况乐观约 14%。
这条 RTL 到 43.9 MHz 就到底了	那是旧架构的上限。第 7 代换掉调度结构后做到 55.56 MHz。
四条消息跑通就算接口没问题	还得在陌生机器上用一条命令复现。CI 第 6 道门干的就是这件事。

我不做的事也写清楚：不亲自写每行 RTL，不盯每个日志，不为了一个更漂亮的数字再加一代。

归属

哪些放开了，哪些留着

开源不是把东西送掉，是把使用权按条款让出去，署名权和文档权留着。三层分开写清楚，比一句「版权所有」有用。

层	许可 / 归属
代码（脚本、驱动、CI 配置）	Apache-2.0。可商用、可修改，保留版权声明与许可文本即可。
硬件（RTL 与 GDSII）	CERN-OHL-P-2.0。可用于商业设计，须保留署名与出处。
文档与作品集（项目报告、验收手册、本 PDF、著作权保留。允许原样转发与引用（须署名 + 原始链接）；不允许修改后署名他人、打包售卖、作为付费课程或培训素材、用于资质投标材料中的虚报。

为什么不把文档也开源

代码可以被人拿去用，方法不能被人拿去当自己的资历。一个人做出来的东西，最值钱的不是文件，是「这件事是他做的」这个事实。

不开公司也能有产权

著作权自作品完成时产生，不需要登记，自然人就是权利人；公开仓库的每一次提交都带时间戳，这就是时间证据。开公司是经营需要，不是拥有作品的前提。

引用格式见仓库 CITATION.cff；署名 1ouxilisi（公众号：欧西1847）。

06 / OTHER WORK

其他在做的

把做到哪一步，一起摊开

只有一件作品会让人以为是运气。所以这张表把别的方向也列出来——但每一行都标了到底做到哪一步，「只想过」的别当真。

领域	做到哪一步 / 证据长什么样
硬件 / 芯片（SHA-256）	闭环：七代迭代，GDS 可交付。证据是本作品集 + 公开仓库 + 外人可跑的验收命令。
新能源车故障自查	已成稿：按故障现象组织的自查手册。证据是成稿文档，读过的人能自己排查。
人形机器人动作包	在跑：MuJoCo 反解出关节轨迹，浏览器里回放验证。证据是可回放的动作序列，未成套。
AI 短剧 / 内容	试过，未成体系。只有样片，没有稳定产出流程。
飞行器	只有想法，没有动手。

这张表怎么用

看哪一行写着「闭环」，就知道我现在能把什么交到你手上；写着「在跑」的，能看过程但还接不了活；写着「只想过」的，就当没写过。

广度不是把五件事都做一半，是知道哪一件已经能交付、哪一件还只是想法——这两件事中间，差的是能不能被人验证。

07 / CONTACT

找我做什么

只做能交付、且能被验证的事

适合找我

- 一件模糊的事，需要有人定目标、切里程碑，然后把它跑成可验证的交付。
- 你的团队已经在用 AI 干活，但没人判断交付真假——我帮你搭验收结构。
- 需要一个「从零到能打开、能检查」的完整案例，而不是一份方案。

不适合找我

- 需要资质签字、需要承担法律责任的鉴定与认证。
- 需要长期驻场，或需要我假装是科班工程师。
- 只需要「看起来很像」的素材，不需要真的能跑。

1ouxilisi

一个人，不是团队。署名之外的任何头衔都不必加。

作品集	`https://1ouxilisi.github.io/sha256-openlane-tapeout/`
项目详情	`.../project.html`
验收手册	`.../VERIFYING.md`
代码	`github.com/1ouxilisi/sha256-openlane-tapeout`
镜像	`gitee.com/beyond-invincibility/sha256-openlane-tapeout`

本页所有数字都能在仓库对应日志里查到。

跑不通的命令，欢迎直接说它是假的。